Hicham Belghachem / Aël Bourasset / Titouan Hibon

Lien vers vidéo YouTube :
https://youtu.be/8ODOPMT4wIo?si=IXIA29tEx1ImswW7

Lien vers Github :

https://github.com/SD-hbelgha/SAE_5.02_No_SQL/tree/main

Introduction

L'objectif de ce document est de résumer tout notre travail et de montrer l'intérêt d'être passé d'une base SQL à une base graphe sous Neo4j. Pour être clair, notre rapport s'organise en quatre parties.

Pour commencer, nous ferons une documentation complète des travaux réalisés. Cette partie reprendra dans l'ordre toutes nos étapes techniques, de la création du MCD et du nettoyage des données sur SQLite, jusqu'à la migration finale.

Puis nous ferons une comparaison des approches relationnelle et graphe et compareront les performances des deux approches.

Ensuite, nous ferons une présentation synthétique des résultats et de nos recommandations. Nous montrerons concrètement ce que nos requêtes nous ont permis de découvrir et pourquoi nous conseillons d'utiliser Neo4j.

Enfin, nous expliquerons simplement les différentes méthodes de migration de données qui existent en informatique. Cela permettra de mieux comprendre et justifier nos choix techniques.

I/ Documentation des travaux réalisés

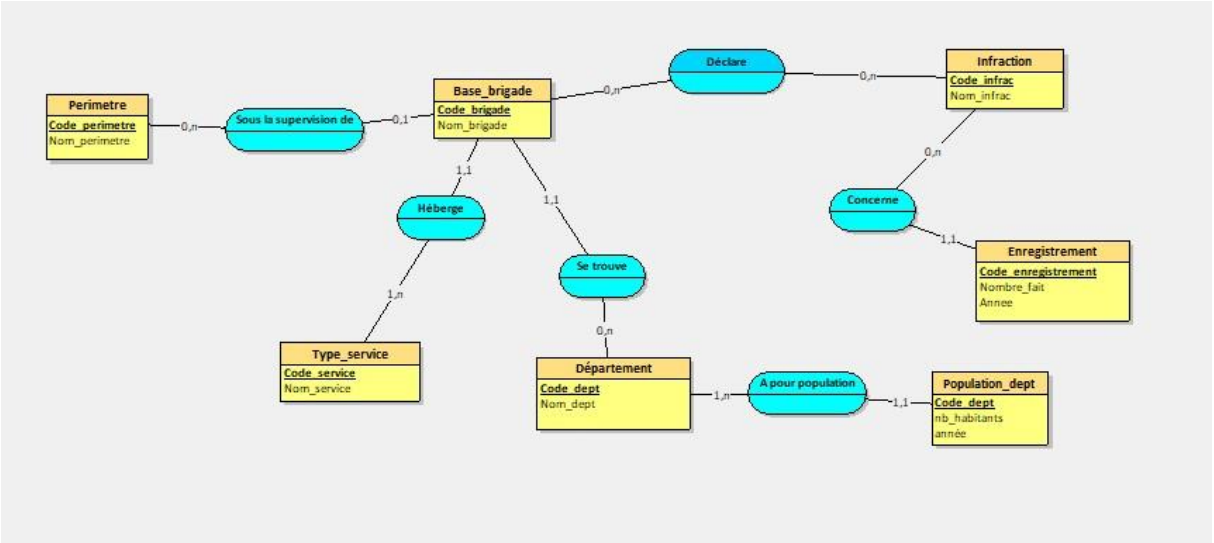
Phase 1 : Modélisation et préparation des données sources

Dans le cadre de ce projet de migration, la première étape a consisté à concevoir un MCD. L'objectif est de savoir comment nous allons structurer nos données pour pouvoir faire des analyses sur les crimes et délits.

Les données initiales nous ont été fournies sous un Excel par années et par services via des feuilles distinctes pour la police nationale et la gendarmerie qui sont-elles-même découpés par années. Nous avons décidé d'ajouter la population totale par département et années (INSEE). Nous avons retraité ce fichier manuellement pour regrouper sur une seule feuille la population totale par département et par année.

Côté MCD nous avons la table « Base_brigade » qui centralise l'essentiel des données car ce sont les brigades qui déclarent les infractions. En identifiant une brigade nous connaissons naturellement le département dans lequel elle opère, mais aussi son service d'appartenance (gendarmerie ou police nationale). Dans le cas

spécifique de la police nationale, cette table nous permet également de déterminer le périmètre d'action de la brigade.



MCD du projet

Ces brigades sont ensuite rattachées à la table Département. C'est à ce niveau précis que s'effectue la jointure avec notre jeu d'enrichissement de l'INSEE, ce qui permet d'associer directement la démographie annuelle à chaque zone.

La table Infraction joue le rôle de référentiel pour tous les types de crimes et délits recensés.

Enfin, la table Enregistrement agit comme la clé de voûte de notre modèle. Elle fait le lien entre toutes les autres tables en attribuant un code unique pour chaque croisement entre un type d'infraction, une année et une brigade.

Phase 2 : Déploiement de la base relationnelle sous SQLite

Une fois le MCD défini, l'étape suivante crée la base de données via SQLite. Pour en arriver là nous avons due commencer par le nettoyage des données brutes, après quoi nous avons pu commencer la structuration des données via SQLite

Les données sources n'étaient pas directement exploitables par un SGBD. Concernant les statistiques criminelles sur Excel, la difficulté résidait dans leur éclatement, notamment les différences de mise en page entre les services de la police et de la gendarmerie ou les feuilles séparé par années par exemple.

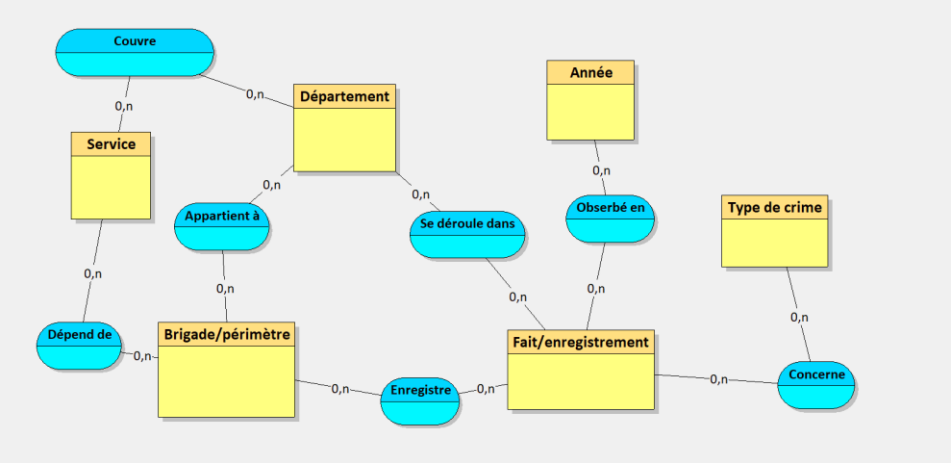
Nous avons donc développé un premier script automatisé en Python « Script_traitement_XLSX.py ». Le script itère sur chaque feuille du classeur et extrait l'année depuis le nom de l'onglet, puis si la feuille concerne la police, il gère un décalage spécifique des lignes d'en-tête (pour capturer les périmètres) qui diffère de la gendarmerie. Ensuite, nous itérons sur chaque ligne et colonne pour créer des

enregistrements des crimes et délits exploitable via DB Browser. Dans la même logique, un second script « Script_traitement_population.py » a été créé pour réduire les données à une seule feuille. Ce script isole l'année, puis supprime les lignes de totaux globaux (comme "France métropolitaine") et force la conversion des valeurs de population en nombres entiers pour éviter les erreurs de typage. Toutes ces données sont ensuite triées et concaténées pour crée notre fichier d'enrichissement final « population_departements.csv ».

À l'aide du fichier « Requetes_creation_tables.sql », nous avons créé l'architecture de notre future base en gardant la structuration présente dans le MCD. Les données ont été importés directement sur DB Browser et idem pour notre .sql. Après avoir liées nos tables à nos données et fait nos jointures, nous avons exporter nos données de SQLite sur « BDD_SQL.db ».

Phase 3 : Migration automatisée vers Neo4j

Après le passage d'Excel à SQLite, l'étape suivante a consisté à migrer l'ensemble de ces informations vers un modèle orienté graphe via Neo4j. Pour préparer cette bascule, nous avons d'abord repensé l'architecture en créant un nouveau schéma visuel « Noeuds&Relations.png ». Dans cette nouvelle logique, nos entités se transforment en Nœuds (Brigade, Département, Infraction) et nos clés étrangères deviennent des Relations (arêtes) reliant ces entités.



Noeuds&Relations.png

Pour réaliser cette migration de manière automatisée, nous avons développé un script sous la forme d'un notebook Jupyter « Script_SQLITE_TO_NEO4J.ipynb ». D'un point de vue technique, ce notebook commence par se connecter directement à notre base source « BDD_SQL.db ». Il utilise des requêtes SQL pour interroger nos tables et scinder nos données en séparant les dimensions (les référentiels des départements, brigades, années…) de la table des enregistrements des crimes. Ces données sont exportées sous la forme de plusieurs fichiers CSV.

Le script prend en charge l'injection dans la base graphe en exécutant des requêtes rédigées en langage Cypher. Nous avons utilisé la commande « LOAD CSV WITH HEADERS » pour lire chaque ligne de nos fichiers. Cela garantit que chaque référentiel (comme un code département précis ou un type de crime) n'est instancié qu'une seule et unique fois sous forme de Nœud, empêchant ainsi la création de doublons lors de l'itération sur des millions de lignes.

En relisant les fichiers CSV liant les identifiants, le programme utilise « MERGE » pour générer les relations entre les Nœuds. Il trace des relations comme « ENREGISTRE », « SE_DEROULE_DANS » permettant de lier un fait criminel à sa brigade, son lieu géographique et sa nature.

Phase 4 : Interrogation du graphe et requêtes métiers

Une fois la migration sur Neo4j terminée, la dernière étape a consisté à confirmer que la base orienté graphe fonctionnait bien. Nous avons donc exécuté une série de requêtes. L'objectif était de s'assurer que notre réseau de nœuds et de relations permettait bien de répondre aux questions métiers du ministère de l'Intérieur.

Nous avons cherché à identifier les 5 infractions les plus fréquentes à l'échelle nationale. Techniquement, cette première requête se contente de traverser la relation directe « CONCERNE » entre les nœuds « Fait » et les nœuds « Infraction », puis d'utiliser la fonction d'agrégation "sum" sur le nombre de faits. Le résultat obtenu de manière quasi instantanée confirme que le calcul des volumes globaux fonctionne.

```
1 MATCH (f:Fait)-[:CONCERNE]->(i:Infraction)
2 RETURN i.nom AS infraction, sum(f.nb) AS total
3 ORDER BY total DESC
4 LIMIT 5;
```

	infraction	total
1	"Autres vols simples contre des particuliers dans des locaux ou lieux publics"	3342691
2	"Vols à la roulotte"	2586068
3	"Autres coups et blessures volontaires criminels ou correctionnels"	2435692
4	"Escroqueries et abus de confiance"	2232757
5	"Cambriolages de locaux d'habitations principales"	2156510

Requête Neo4j

Nous avons également exécuté une requête pour identifier le département le plus touché, mais uniquement mais uniquement pour les gendarmeries. La requête part du département, interroge les brigades qui y appartiennent, filtre celles qui dépendent strictement du service de la gendarmerie et redescend enfin vers les faits enregistrés et leurs années.

```
MATCH (b:Brigade)-[:DEPEND_DE]->(:Service {id:2})
MATCH (b)-[:APPARTIENT_A]->(d:Departement)
MATCH (b)-[:ENREGISTRE]->(f:Fait)
MATCH (f)-[:OBSERVE_EN]->(a:Annee)
WITH d, a.valeur AS annee, sum(f.nb) AS total_annuel
WITH d, avg(total_annuel) AS moyenne
ORDER BY moyenne DESC
LIMIT 1

MATCH (d)<-[:APPARTIENT_A]-(b:Brigade)-[:DEPEND_DE]->(:Service {id:2})
MATCH (b)-[:ENREGISTRE]->(f2:Fait)-[:OBSERVE_EN]->(a2:Annee)
WITH d.code AS dep, b, a2.valeur AS annee, sum(f2.nb) AS total_annuel_brigade
WITH dep, b, avg(total_annuel_brigade) AS moyenne_brigade
RETURN dep, b.nom AS brigade, moyenne_brigade
ORDER BY moyenne_brigade DESC
LIMIT 1;
```

dep	brigade	moyenne_brigade
"38"	"CGD MEYLAN"	8353.0

Requête Neo4j

II/ Analyses de l’approche relationnelle et graphe

Comparaison des approches relationnelle et graphe

L’approche relationnelle repose sur une organisation des données sous forme de tables reliées entre elles par des clés étrangères. Ce modèle est très efficace pour stocker des données structurées et effectuer des opérations classiques comme des agrégations ou des filtrages. Cependant, lorsque les analyses nécessitent de nombreuses relations entre entités, les requêtes deviennent rapidement complexes à cause des multiples jointures nécessaires entre les tables. À l’inverse, l’approche orientée graphe modélise directement les entités sous forme de nœuds et les relations sous forme d’arêtes. Les relations deviennent des éléments de première classe dans la base de données. Cette structure permet de représenter naturellement des systèmes fortement connectés, comme dans le cas de l’analyse des crimes reliant départements, brigades, périmètres, infractions et années. Les requêtes sont généralement plus lisibles et plus proches de la logique métier.

Comparaison des performances entre les bases relationnelles et graphes

En termes de performances, les bases relationnelles sont très optimisées pour les requêtes analytiques classiques sur de grands volumes de données, notamment pour les agrégations ou les filtrages sur des colonnes indexées. Cependant, lorsque

les requêtes impliquent de nombreuses relations entre entités, les jointures peuvent devenir coûteuses en temps de calcul. Les bases orientées graphe, comme Neo4j, sont conçues pour optimiser la navigation entre les relations. Les relations étant stockées directement entre les nœuds, les traversées de graphe sont souvent plus rapides et plus efficaces que des séries de jointures dans un modèle relationnel. Dans ce projet, les analyses impliquant plusieurs niveaux de relations (par exemple entre brigades, faits, départements et types d'infractions) sont plus simples à exprimer et potentiellement plus performantes dans le modèle graphe, car elles exploitent directement la structure relationnelle des données.

III/ Présentation synthétique des résultats et des recommandations

Synthèse des Résultats Analytiques

L'exploitation de la base orientée graphe révèle des tendances claires sur la criminalité en France entre 2012 et 2021. À l'échelle nationale, les atteintes aux biens dominent largement. L'infraction la plus fréquente concerne les « Autres vols simples contre des particuliers dans des locaux ou lieux publics », suivie de près par les « Vols à la roulotte ».

L'analyse géographique montre que les départements urbains très peuplés, comme Paris (75), le Nord (59) et les Bouches-du-Rhône (13), concentrent le plus grand nombre brut d'infractions. À l'inverse, des territoires ruraux ou ultramarins comme Wallis-et-Futuna (986) et la Lozère (48) affichent les volumes les plus faibles.

Le ciblage par juridiction permet d'isoler des secteurs sous tension. En zone de compétence de la Gendarmerie Nationale, l'Isère (38) enregistre la moyenne d'infractions la plus élevée. Côté Police Nationale, le périmètre de la « DSPAP » à Paris ressort comme la zone d'action la plus impactée.

Enfin, l'intégration des données démographiques de l'INSEE prouve sa valeur stratégique. Le calcul du taux d'infractions pour 1000 habitants corrige le biais lié au volume de population. Ce ratio met en évidence la pression criminelle réelle dans des départements comme Paris (75) et la Guyane (973).

Pour plus de détails voir « Analyses_NEO4J.pdf »

Recommandations

À la suite de ces travaux, plusieurs recommandations s'imposent pour le ministère de l'Intérieur.

Il est d'abord conseillé de généraliser l'utilisation des bases orientées graphes pour les équipes. Ce modèle surpasse le SQL traditionnel pour l'investigation de réseaux de données fortement connectées.

Concernant l'allocation des effectifs et des budgets territoriaux, le pilotage doit privilégier le taux de criminalité pour 1000 habitants plutôt que le volume brut des délits. Cet indicateur garantit une analyse équitable entre les départements.

IV/ Les différentes méthodes de migration qui existent

La migration de données consiste à transférer des données d'un système vers un autre tout en conservant leur cohérence et leur structure logique. Dans le cadre de ce projet, il s'agit de migrer des données d'une base relationnelle vers une base de données orientée graphe comme Neo4j.

Plusieurs méthodes de migration existent :

1. La migration ETL (Extract, Transform, Load)
C'est la méthode la plus utilisée. Elle consiste à :

- Extraire les données depuis la base source,
- Transformer les données pour les adapter au nouveau modèle (ici transformer les tables relationnelles en nœuds et relations),
- Charger les données dans la base cible.

2. La migration par scripts automatisés
Cette méthode consiste à utiliser des scripts (souvent en Python ou autre langage) pour lire les données de la base relationnelle, les transformer et les insérer dans la base graphe via des requêtes Cypher.

3. La migration via des outils spécialisés
Certains outils permettent de faciliter la migration entre différents systèmes de bases de données. Ils automatisent une partie de l'extraction, de la transformation et du chargement des données.

Ajout de nouvelles données dans les bases de données

Dans une base de données relationnelle, l'ajout de données se fait à l'aide de requêtes SQL (INSERT). Les nouvelles informations doivent respecter la structure des tables et les contraintes définies (clés primaires, clés étrangères, etc.).

Dans une base de données orientée graphe, comme Neo4j, les données sont ajoutées sous forme de nœuds et de relations à l'aide du langage Cypher (CREATE ou MERGE). Cela permet d'ajouter facilement de nouvelles entités ou relations sans modifier fortement la structure existante.

Les deux systèmes permettent l’ajout de nouvelles données, mais les bases graphes offrent généralement plus de flexibilité pour représenter et enrichir les relations entre les informations.